

BAGEL: An Approach to Automatically Detect Navigation-Based Web Accessibility Barriers for Keyboard Users

Paul T. Chiou
University of Southern California
USA
paulchio@usc.edu

Ali S. Alotaibi
University of Southern California
USA
aalotaib@usc.edu

William G. J. Halfond
University of Southern California
USA
halfond@usc.edu

ABSTRACT

The Web has become an essential part of many people's daily lives, enabling them to complete everyday and essential tasks online and access important information resources. The ability to navigate the Web via the keyboard interface is critical to people with various types of disabilities. However, modern websites often violate web accessibility guidelines for keyboard navigability. In this paper, we present a novel approach for automatically detecting web accessibility barriers that prevent or hinder keyboard users' ability to navigate web pages. An extensive evaluation of our technique on real-world subjects showed that our technique was able to detect navigation-based keyboard accessibility barriers in web applications with high precision and recall.

CCS CONCEPTS

• **Human-centered computing** → **Accessibility systems and tools**; • **Software and its engineering** → *Software maintenance tools*; **Software testing and debugging**.

KEYWORDS

Web Accessibility, WCAG, Software Testing, Keyboard Navigation

ACM Reference Format:

Paul T. Chiou, Ali S. Alotaibi, and William G. J. Halfond. 2023. BAGEL: An Approach to Automatically Detect Navigation-Based Web Accessibility Barriers for Keyboard Users. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*, April 23–28, 2023, Hamburg, Germany. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3544548.3580749>

1 INTRODUCTION

Companies create web applications to reach a broad audience and enable customers to access services and information. The ability to access these daily-life resources with ease is important for everyone, especially for the approximately 15% of the world's population with disabilities [20]. Studies show that as much as 51% of the disabled population rely on the Web as a societal lifeline [6]. Unfortunately, sites on the Web pose many accessibility challenges for users with disabilities. As of 2022, less than 4% of the top million homepages on the Web meet the most widely used accessibility standards [37].



This work is licensed under a Creative Commons Attribution-NonCommercial International 4.0 License.

CHI '23, April 23–28, 2023, Hamburg, Germany
© 2023 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9421-5/23/04.
<https://doi.org/10.1145/3544548.3580749>

When web applications are poorly designed, they create barriers that threaten the goal of universal accessibility.

Keyboard navigation is one of the most important aspects of web accessibility that is often overlooked among web developers. Web users with limited vision or motor movement that cannot utilize a point-and-click or touch-operated device rely on keyboard commands to navigate a web application's User Interface (UI) via Assistive Technologies (ATs). However, many of these ATs present information in web pages under the assumption that the keyboard navigation follows a linearized structure that is perceivable, which is often not the case. In modern web applications, changes and updates generated by JavaScript are not always presented to keyboard-based users interacting with the Web through ATs. Keyboard navigability is one of the most common and prevalent accessibility issues [22, 25, 28]. Studies show that as many as 29% of popular websites violate ways to help users navigate or find content/functionality, and 9% lack structural indicators for helping users orient themselves to determine where they are in a web page [75]. In this paper, we call the manifestation of these issues *Keyboard Navigation Failures (KNFs)*, which are, broadly, failures that prevent the user from intuitively navigating the web app's UI during keyboard interaction.

Detecting keyboard navigability issues is a challenging task due to the complex nature of modern client-side web applications. For example, identifying KNFs requires observing specific behaviors when navigating the UI via keyboard interaction. Simply analyzing the HTML source code or the rendering of the web page and its properties via static-based analysis techniques is not enough to expose the KNFs. In this paper, we present a novel approach for automatically detecting KNFs by using dynamic web crawling techniques to understand the navigation and the visual semantics of a web page's UI. An overview of our approach is shown in Figure 1. The approach consists of two main parts. The first part builds models that capture the possible behaviors of a keyboard-based user's run-time interactions with the web page (Section 4). The second part analyzes these models to identify the navigation and visual behaviors that would represent KNFs (Section 5).

We implemented our approach as a prototype, BAGEL, to evaluate how well our technique could detect KNFs in real-world web applications. Our evaluation shows that BAGEL was able to accurately detect KNFs with an average F1 score of 90%. The key contributions of this paper are:

- (1) The first formalization and modeling of Keyboard Navigation Failures (KNFs) as defined by the W3C.
- (2) A fully automated KNF detection technique that outperforms state-of-the-art on real-world web applications.

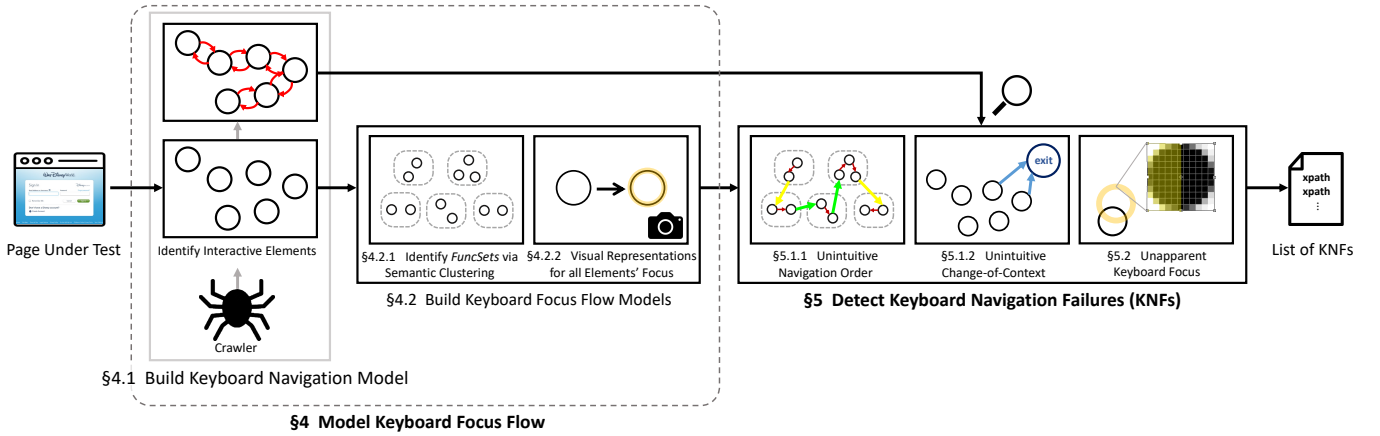


Figure 1: A flowchart that shows the overview of our approach. The approach takes a Page Under Test (PUT) as the input and builds (1) a keyboard navigation model (Section 4.1) and (2) models that characterize keyboard focus (Section 4.2). All these models are later used in analyses that detect KNFs (Section 5). The output of the approach is a list of KNFs for the PUT.

- (3) A study on a set of 20 real-world web pages that shows our approach is accurate in detecting KNFs.

2 BACKGROUND AND MOTIVATION

Keyboard usage is one of the foundational requirements for web accessibility and it is reflected in W3C Web Accessibility Initiative (WAI)’s Web Content Accessibility Guidelines (WCAG) 2.1. When interacting with a web page using a point-and-click device, every web control element, (i.e., a hyperlink) is available to be accessed. However, when interacting with a web page using a keyboard-based device, such access is sequential and depends on the keyboard navigation sequence. For example, during keyboard navigation, the user executes the **TAB** command to advance the keyboard cursor to the next (or **SHIFT+TAB** to go back to the previous) control element. The actuation keys (**SPACE** or **ENTER**) are then used to activate the element that is currently in focus. Similarly, the **ESC** key is used to exit an active user prompt or dialog. Due to this sequential navigation, both the order of the sequence and the appearance of the keyboard cursor are crucial for keyboard-based users.

Navigation order is a significant aspect of a UI, especially for different ATs (e.g., parallel visual scanning, linear exploration using a screen reader, and serial navigation using a switch interface). However, design and implementation mistakes by web developers can hinder keyboard-based users’ ability to effectively navigate the UI. In this paper, we address the two general types of KNFs: **Unintuitive Navigation Sequence** and **Unapparent Keyboard Focus**, which are defined under WCAG Success Criteria (SC) 2.4.3, 2.4.7, 3.2.1, and 3.2.2 [46–49]. These failures are all of the general keyboard navigation issues defined by WCAG that ensure that keyboard operability is intuitive and predictable. We describe each KNF next in detail.

2.1 Unintuitive Navigation Sequence

2.1.1 Unintuitive Navigation Order. An unintuitive order arises when the keyboard navigation is not sequentially consistent or intuitive with the meaning of the content. This can occur when

developers (1) explicitly assign elements’ `tabindex` attribute with values greater than 0, or (2) reorder contents using the `CSS float`¹, `CSS Grid`¹, or `CSS Flexbox Layout`¹ modules to format elements [4, 11], making the navigation order inconsistent from their visual order with respect to the Document Object Model (DOM)². This creates an unexpected tab order, making the page less intuitive, and can skip certain elements entirely to disorient and confuse screen reader users.

An example of this is in Figure 2’s *Disney World* log-in page, where the navigation order is depicted by the bulleted numbers. In this example, the *Password* text-field (❷) and the “*Forgot password?*” link (❸) are visually adjacent to each other because their semantics are both associated with functionalities related to a user’s password. This type of visual organization is the intuitive way developers design web applications. However, these two controls are not adjacent to each other in keyboard navigation. As a result, when the keyboard navigation advances from ❶, ❷, ❸, the functional semantics of the elements becomes inconsistent. The same applies to the “*Remember Me*” checkbox (❺) with respect to the *Email* and *Password* text-fields. From a blind user’s perspective, when the controls with similar semantics are not grouped together, they could falsely believe the web form is at its end. For example, when they may navigate to the submit button (❹) without the awareness that there is still a “*Remember Me*” checkbox (❺).

As outlined in WCAG [42, 43], a meaningful sequence should follow a linear visual flow of the page (e.g., header to content to footer).

2.1.2 Unintuitive Change-of-Context. Another scenario is when the web page changes its context (or focus behavior) unexpectedly during keyboard navigation. This “Change-Of-Context” KNF occurs when a UI/web functionality is actuated upon receiving keyboard focus or keyboard input without user awareness. An example

¹CSS page layout techniques that allow developers to take elements contained in a web page and manually control where they’re positioned.

²The DOM is the data representation of the objects that comprise the structure and content of a web page that is displayed in the browser.



Figure 2: An example of *Unintuitive Navigation Order* type KNFs from Disney’s log-in web page that demonstrates when the keyboard focus jumps across different functionalities from different parts of the page. The numbers annotate the navigation sequence.

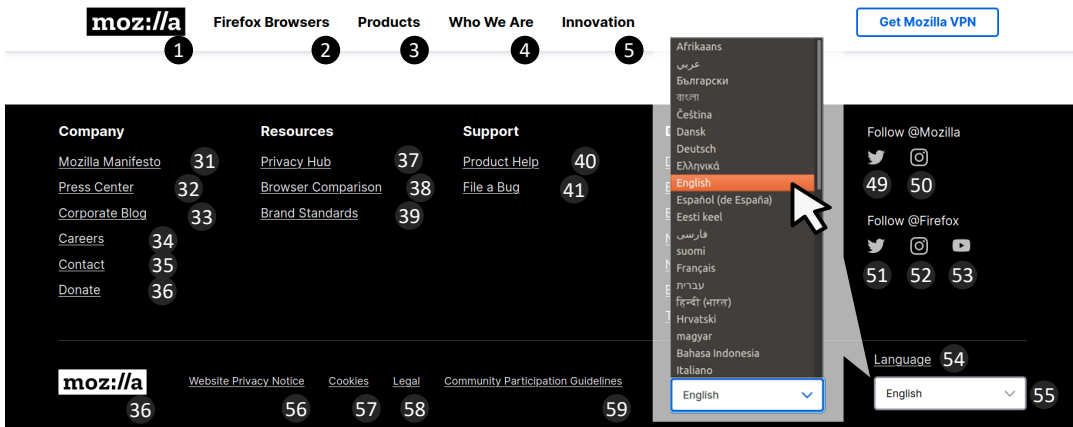


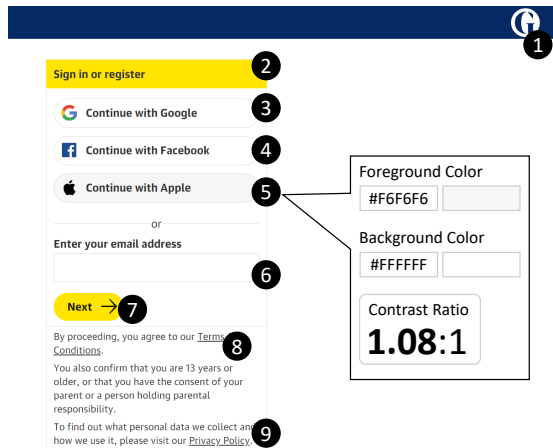
Figure 3: An example of *Unintuitive Change-Of-Context* type KNFs from the footer section of Mozilla’s web pages that demonstrates a drop-down menu that causes the keyboard navigation to unexpectedly navigate away from the page without user confirmation.

resides in Figure 3 Mozilla’s footer language selection drop-down component (54). When this language drop-down is in focus and the user presses $\uparrow$ or $\downarrow$, the drop-down’s scripting directly activates the “next” (or previous) language selection and navigates to a new page of the selected language without the user’s awareness. This can be disorienting, especially for a keyboard user because they were not advised about, nor confirmed the action to leave the current web page. According to WCAG Techniques G107 [26], an ideal implementation of this language drop-down is to be able to navigate across all of the available languages via $\uparrow$ and $\downarrow$ and activate the selected one via **ENTER** (like how a mouse user would hover over their desired selection and click as shown in the figure).

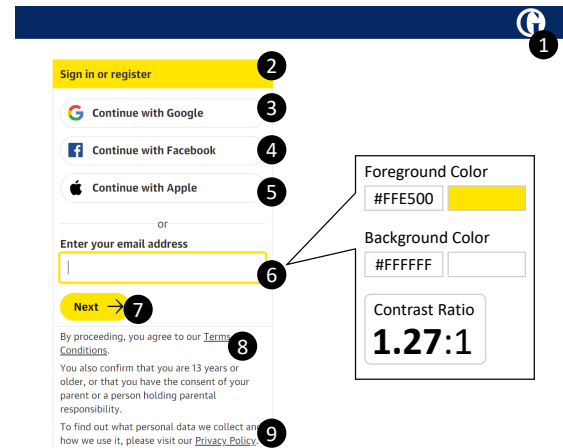
As outlined in SC 3.2.1 and 3.2.2, unexpected changes of context can make interactive content less predictable and disorienting for keyboard-based users with visual disabilities or cognitive limitations so that they are unable to use the content.

2.1.3 Challenges to Detect Unintuitive Navigation Sequence. A naive way to detect Unintuitive-Nav-Order KNFs is to simply examine elements whose `tabindex`³ attributes are set to a positive integer that overrides the default tab order. However, three challenges make this naive approach insufficient for our problem domain. First, a positive `tabindex` may not necessarily lead to inconsistent navigation – it may even lead to false-positives on accessible components implemented with the “roving tabindex” JavaScript techniques that provide focus by dynamically altering the `tabindex` attribute between -1 and 0 within a group of controls [10]. Second, there may be more than one “correct” navigation sequence, which itself can be dynamically altered via scripting during run-time. Third, it is difficult to examine and understand the semantics of a UI component’s (e.g., drop-down list) JavaScript code that is responsible for event handlers. This is because the scripts may be dynamically

³`tabindex` global attribute allows an element to receive focus and indicates where it participates in sequential keyboard navigation via the Tab key.



(a) “Continue with Apple” button’s faint gray highlight



(b) “Enter your email address” input’s yellow outline

Figure 4: An example of Unapparent-Focus type KNFs from *The Guardian*’s log-in web page that demonstrates two cases where different elements’ keyboard focus indicators lack sufficient color contrast for the user to identify the location of the keyboard cursor.

loaded and not always available at the start of a web application. Scripting behaviors may also change as keyboard interaction enters different parts of the UI. These challenges make it difficult to detect Unintuitive-Nav-Order KNFs using traditional static program analysis techniques that only scan the HTML source code of the web page because interactive behaviors are not revealed in the page’s source codes.

2.1.4 Closest Related Work to Unintuitive Navigation Sequence. The closest work related to Unintuitive Navigation Sequence is industrial web accessibility auditing tools that can detect general WCAG violations [18, 29–31, 36]. These tools can identify patterns in the HTML source code for missing page title, missing form labels, out of order or skipped heading levels (e.g., level <h3> heading structured before level <h2>), the lack of “skip navigation” shortcut links that allows ease of navigation, and the use of <table> to structure the visual layout rather than using CSS structural layout to help orient users within the content. While these techniques are capable of detecting simple layout inconsistencies, they cannot accurately detect KNFs (as we show in Section 6) that are dynamic in nature, because they are not capable of interacting with the web pages to reveal the behaviors of Unintuitive-Nav-Order KNFs.

KAFE [63] proposed a dynamic-based technique to detect UI elements that are not accessible to keyboard users [44, 45]. It does not measure the consistency of keyboard navigation sequences and does not have a sense of the contextual information about a web page’s content. Hence, it is not able to detect Unintuitive-Nav-Order KNFs.

2.2 Unapparent Keyboard Focus

A focus indicator [3] allows keyboard users to know their current location on a web page. By default, a basic focus indicator is provided by web browsers and is shown as an outlined border around the focused element (a focus ring). Form fields also show a vertical bar (text cursor) inside the field during text input. An unapparent

focus type KNF occurs when the keyboard focus mechanism is not present or observable at all times, preventing the user from visually locating the interactive element that is ready to be activated or manipulated. More subtle failures occur when the surrounding background is visually similar to the custom focus style on a link or other controls or if their colors have insufficient contrast to be clearly noticed. Figure 4 shows *The Guardian*’s login page. When the keyboard focus is navigated to the “Continue with Apple” button (5), the button is highlighted with a faint grayish color (#F6F6F6) that is not clearly visually distinguishable from its non-focused counterpart (with a color contrast ratio of only 1.08 : 1). Similarly, when the focus is on the “Enter your email address” text-field (6), the color contrast between the yellow focus ring (#FFE500) and the white background (#FFFFFF) is only 1.27 : 1.

As outlined in SC 2.4.7 any keyboard operable user interface is required to have a focus indicator that is visible. A corollary is defined [15] that requires the visual presentation of non-text (such as the focus indicator) to have a minimum contrast ratio of 3 : 1 against its adjacent color(s).

2.2.1 Challenges to Detect Unapparent Keyboard Focus. A naive way to detect Unapparent-Focus KNFs is to examine the CSS applied to a web page to identify styling that removes or renders the visual focus indicator non-visible. However, this is troubled by two challenges. First, the focus indicator can be achieved in many different ways other than the CSS outlines⁴ and borders⁴ properties. For example, developers can change the presentation of UI components with inverted colors, highlights, or use JavaScript to animate the focus indicator with transitions from one element to another. Scripting can also be used to remove focus indicators upon receiving focus. This makes no determinant way to identify the visibility of focus indicators by examining specific CSS properties.

⁴CSS outline and border properties allow developers to specify a line that is drawn around an element and the style of the element’s border.

Second, the intricacies of dynamic layout rendering can behave in unintuitive ways and be affected by surrounding elements. For example, run-time *z-index*⁵ or *outline-width*⁶ behaviors can cause the focus indicator of an element to be cut off, exceed the bounds, or be overlapped by other elements. Visual rendering becomes more complicated to examine when elements are rendered over background-images or background with opacity that are visually similar for the focus indicator to be clearly noticed. The dynamic layouts can also change as the user interacts or navigates the web page. These challenges make it difficult to detect Unapparent-Focus KNFs using traditional static program analysis techniques that rely on simply analyzing the rendered DOM of the web page when the page loads.

2.2.2 Closest Related Work to Unapparent Keyboard Focus. Several approaches have attempted to address UI navigation to easily identify the cursor location [67] and to efficiently select targets on the screen. Object Pointing [70] modifies the presentation of the mouse cursor based on Fitts's law [80] by overriding the default cursor behavior to jump from one element directly to another. SALEM [53] focuses on improving touch screen navigation for people with motor impairments by automatically repairing the insufficient sizes of touch targets. Similarly, Touch Guard [100] enhances UI elements' touch areas with screen magnification to enlarge and disambiguate the bounds between multiple targets. While these techniques may improve users' ability to interact with applications via mouse or touch-based interactions, they do not model the keyboard focus and therefore are not able to detect Unapparent-Focus KNFs.

3 RELATED WORK

There is extensive literature on detecting web content accessibility or usability issues. Here, we present other related works that cover similar concepts of accessibility or UI navigation, but do not directly address KNFs.

3.1 Techniques to Audit Web Accessibility

General strategies that address web accessibility include the detection of inaccessible layout properties and violations of accessible semantic structures. VizAssert [88, 89] uses formal verification methods to attempt detection and repair of inaccessible layout properties. It makes an abstraction on the visual properties of the web page and then, uses efficiently-solvable SMT queries to automatically assert violations of visual layout and color-contrast consistencies. AXERAY [55] is an automated approach that infers semantic groupings of elements across various regions of a web page to test if these elements violate their WAI-ARIA roles' semantic structure. These techniques specifically focus on accessibility relating to layout properties.

Acceptance web testing techniques, such as Pyccuracy [95] and Cucumber [32] have the potential to detect KNFs, but are not fully automated and require developers' manual intervention. For example, these techniques allow developers to manually write or use

scenario-based test cases to check keyboard accessibility requirements but the developers must anticipate the problems that may occur for each web page.

3.2 Usability of General UI Navigation

Navigation is a widely explored concept in usability research that investigates effective ways for users to interact with UIs. Such research spans across mobile, wearable, embedded, desktop systems [94], as well as websites [91]. Other work on usability [56] has focused on improving web navigability for low-vision users via screen magnification [57, 59, 60, 77, 99]. To mitigate the drawbacks of disorientation caused by magnification, *overview+detail* interface designs [71, 74, 78] were inspired to provide an overview of web pages with respect to their surrounding context for users to better locate the contents of interest. While these approaches can improve web accessibility from a visual perspective, they do not focus on the operability of a web page's UI via the keyboard.

Related work has used adaptive systems to improve performance in web navigation [73] related to tree structures [62], menu selection [66], and effective button placements [64] to reduce visual clutter in form design [76, 87]. Harms et al. [72] proposed a design space analysis to support usability engineering that focuses on web forms. SUPPLE++ [69] automatically personalizes web interfaces by adapting UI elements to meet the accessibility requirements for individuals with visual and motor disabilities. However, such adaptation techniques focus mainly on navigation via point-and-click devices such as a mouse, which does not relate to UI navigation issues that cause KNFs.

The understanding of a web page's semantic structure is a classic and long-standing research problem in addressing web navigability [61, 85, 98, 101]. Sanoja et al [93] proposed web segmentation techniques to identify segments in web pages based on visual presentation style and spatial locality that are organized into semantic hierarchies. Furnas [68] employed a graph-based approach to model the relationship between web views and their outgoing navigation links by minimizing feasible navigation paths. Perona et al [90] used supervised machine learning techniques to detect web navigation problems based on client-side interaction data. Liu [79] focused on web mining to obtain real navigation features to learn anomalous attributes' values to identify potential indicators for detecting navigation problems. Although these approaches have the potential to identify navigation inconsistency or to spot incorrect accessibility metadata used when navigating the web page via ATs, they do not fundamentally address KNFs.

A study by Alshayban et al. [54] examined accessibility issues that impact the *Traversal Order* of UI elements during linear navigation (e.g., via screen readers). However, the work only applies to the Android UI. Latte [92] and ATARI [102] are techniques that execute assistive services to detect navigation inconsistencies. However, Latte requires developers to manually provide test cases that are strictly tailored for specific test subjects and ATARI assumes the availability of a specific UI that is manually provided by the developers. These techniques can improve usability by debugging the inaccessibility that prevents a UI element and its functionality from being available to screen-reader users via TalkBack navigation. Though, it only works on Android applications and cannot be applied to keyboard accessibility in general.

⁵CSS *z-index* property that defines the order of overlapping elements. Those with a higher index will be placed on top of those with a lower index.

⁶CSS *outline-width* property sets the thickness of an element's outline. An outline is a line that is drawn around an element.

4 MODELING KEYBOARD FOCUS FLOW

The goal of our approach is to automatically detect KNFs in web applications and identify the underlying faulty behaviors responsible for the failures. To address the aforementioned challenges, a key capability required to detect KNFs is to understand how the keyboard focus behaves in real-time when a keyboard user interacts with a web page. Our approach uses a dynamic crawling technique to interact with a web page by programmatically simulating user keyboard actions through the browser's keyboard API (e.g., executing **TAB** or **SHIFT+TAB**) and retrieving the active element that is receiving focus for every such interaction. The way the keyboard focus indicator visually transitions between interactive elements is then translated into a graph-based abstraction representing the model that we will analyze. The approach defines a model, called the Keyboard Focus Flow Graph (KFFG) that captures (1) the keyboard navigation allowed by the PUT from the keyboard-based users' perspective (Section 4.1), and (2) the ways the PUT is semantically structured and how the keyboard focus visually appears in the UI (Section 4.2). These models are used later to detect the undesirable behaviors caused by KNFs.

4.1 Building the Keyboard Navigation Model

4.1.1 Defining the Keyboard Focus Flow Graph. The keyboard navigation and focus-related properties of a PUT are represented by the Keyboard Focus Flow Graph (KFFG). The Keyboard Focus Flow Graph (KFFG) is formally represented as a graph $\langle V, V_s, E, V_f, v_0 \rangle$, where V is the set of nodes that consist of all visible elements in the UI, $V_s \subseteq V$ represents the set of all visible elements that are also interactive, E is the set of directed edges that represents the corresponding navigation flow among the interactive elements in V based on their keyboard focus, V_f represents the sets of Functionally Similar Elements (*FuncSets*) that are characteristically/visually similar in the PUT, and v_0 is the entry node of the KFFG.

A node $v \in V_s$ in the KFFG represents a UI element that is interactable (able to receive focus) to the keyboard that provides functionality with which the user can navigate, enter text, etc. V_s includes all the native control elements such as HTML links $\langle a \rangle$, inputs and form controls $\langle button \rangle$, $\langle input \rangle$, $\langle select \rangle$, and $\langle textarea \rangle$ [14] as well as non-native control elements that have been customized with interactive characteristics. (e.g., elements with a `tabindex` property set with a non-negative integer, or elements binded with keyboard interactive events). Each node v is defined as a tuple $\langle R, I, I', x \rangle$, where R is the Minimum Bounding Rectangle (MBR)⁷ that encloses v in the PUT, I and I' are the visual representations of R the way they are rendered in the browser before and after v receives focus, and x is the XPath representing v . R is defined as a tuple $\langle x_1, y_1, x_2, y_2 \rangle$ where the coordinates (x_1, y_1) and (x_2, y_2) represent the upper left corner and the lower right corner of v . I' represents how the keyboard focus indicator is visually presented to the user when the focus is on v . The node $v_0 \in V_s$ represents the entry node of the graph. This is the element in the page that is the starting point for keyboard-based navigation.

A directed-edge $e \in E$ is defined as a tuple $\langle v_s, v_t, \phi \rangle$ indicating that the browser's keyboard focus shifts from a source node v_s to

a target node v_t by pressing keystroke ϕ while v_s is in focus. We include all standard keystrokes for web UI interaction where ϕ can be from the set of navigation keystrokes $\{\mathbf{TAB}, \mathbf{SHIFT+TAB}\}$, selection keystrokes $\{\uparrow, \downarrow, \leftarrow, \rightarrow\}$, actuation keystrokes $\{\mathbf{ENTER}, \mathbf{SPACE}\}$, and dismiss keystroke $\mathbf{ESC}$ [16]. In addition to detecting focus shifts, we detect any attempt for the page to navigate away to a different URL by instrumenting the `onbeforeunload`⁸ JavaScript event. For such occurrences, we construct a forward edge e from the originating node to a special node added to the KFFG called v_{ext} . (e.g., $\langle v_{link}, v_{ext}, \mathbf{ENTER} \rangle$).

4.1.2 Constructing the Keyboard Focus Flow Graph. The KFFG model first builds the node set V by rendering the PUT in the browser and then analyzing its DOM to identify each unique HTML element. Each node is uniquely identified by its XPath in the DOM. The entry node of the graph v_0 is then identified by being the first element to receive focus.

The edge set E is then built by dynamically exploring the client-side UI to identify the possible keyboard navigation. The high-level intuition of this process is to iterate over all the interactive elements in the PUT (i.e., V_s) and execute all the keyboard operations (i.e., Φ) on each element to identify the resulting keyboard navigation. A challenge for dynamic UI exploration is its completeness. For example, during the crawling process, the keyboard focus may be obstructed by navigation behaviors, such as inaccessible custom widgets that trap navigation. This can halt the edge creation process and cause parts of the UI to be unexplored. Our technique overcomes this challenge by starting navigation at each possible interactive node, which skips anomalous navigation behaviors (such as loops) that may obstruct the exploration and ensures there will be no unexplored UI components in the PUT.

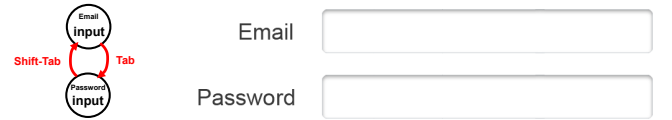


Figure 5: A simple subgraph of the KFFG with its corresponding UI of the web page. The subgraph consists of two adjacent nodes (interactive input elements in the UI) where the keyboard focus can be navigated to each other via **TAB and **SHIFT+TAB**.**

4.2 Building the Models to Capture Keyboard Focus Flow

While the keyboard navigation model provides us with information on how a keyboard user navigates around the UI, it alone cannot be used to detect KNFs. Our approach further builds models that contain (1) information about a web page's content that is needed to detect when the existing focus navigation is out of order. and (2) visual representations of the elements and their analyses needed to detect when the keyboard focus is visually absent during keyboard navigation.

⁷MBR refers to the smallest rectangle which contains the entire element, including its CSS padding and border-width areas around the element.

⁸`onbeforeunload` event occurs when the document is about to be unloaded. It is fired when the user is about to leave the current web page.

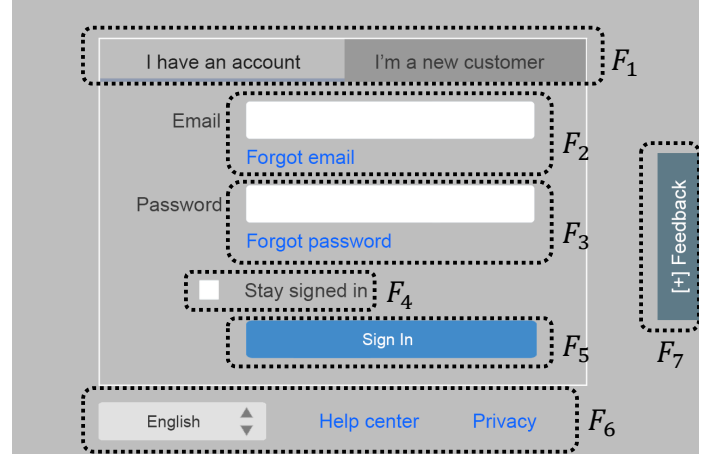
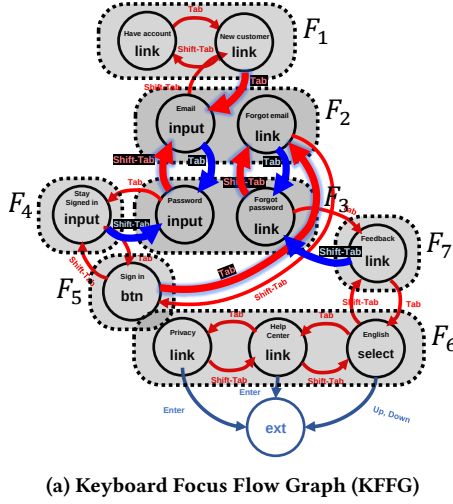


Figure 6: A visual depiction of the KFFG and its corresponding UI of the web page. This example demonstrates two semantical clusters ($FuncSets F_2$ and F_3) in a web page's UI that violate the intuitive navigation order assumptions of our approach.

4.2.1 Identify Similar Page Contents via Semantic Clustering. In order to give the notion of how intuitive the navigation is for the keyboard users (which we will discuss more in Section 5.1.1), the approach employs segmentation on the web page's content information to analyze the navigation. This is done by grouping the elements in V into $FuncSets (F_1, \dots, F_n \in V_f)$ that contain visually related elements in the web page. The goal of this step is to provide an abstraction of the way the web page's content information is structured and presented. Our observation is that related items in the PUT often exhibit consistency in visual presentation style and spatial locality that is used to depict the way users interact and navigate the web page. For example, a set of header menu, footer navigation, social media icons, or form elements that are spatially grouped together tend to be semantically oriented within the content of navigation. Our approach computes the visual similarity and DOM information similarity based on a distance function that uses several metrics such as matching the elements' width, height, alignment, locality, as well as the similarities of their XPath, CSS properties, tag-name, and class and text attributes. The process then uses a density-based clustering technique (DBSCAN) [81] that puts each element into only one cluster (i.e., hard clustering). Each element $v \in V$ belongs in exactly one of the n clusters ($F_i \subseteq V$) and all clusters are disjoint subsets of V .

4.2.2 Visual Representations of the Keyboard Focus Indicators. To understand whether the keyboard cursor (focus indicator) is apparent to keyboard users, the approach captures a set of visual representations of every node $v \in V$ as the way they appear in a browser. This is done by capturing a screenshot that is cropped to the nodes' R region both before (I) and after (I') v is set to focus. Note that I and I' are identical in dimension. The cropping process includes a custom adjustable pixel radius as padding to ensure thick focus rings are not cut off. This visual focus model allows our approach to retain the exact way the focus would appear to a keyboard user during navigation.

5 DETECTING KEYBOARD NAVIGATION FAILURES

To perform the detection of Keyboard Navigation Failures (KNFs), the approach analyzes the keyboard navigation model that is constructed in Section 4.1 as well as the models that characterize keyboard focus constructed in Section 4.2.

5.1 Detecting Unintuitive Navigation Sequence

To detect **Unintuitive Navigation Sequence** type KNFs, whose behaviors are described in Section 2.1, the goal is to identify certain keyboard navigation patterns that represent KNFs. Recall that this general type of KNF can be either: Unintuitive-Nav-Order or Change-Of-Context failures. We explain the detection of each in detail in the following subsections.

5.1.1 Unintuitive Navigation Order. Recall that this KNF occurs when the navigation focus order is inconsistent with the determined reading order or the visual presentation of the web page [13] (e.g., the focus appears to jump around randomly across different sections of the PUT). To detect this KNF, the approach analyzes the keyboard navigation that is represented by the edges in the KFFG with respect to the $FuncSets$. The intuition is that the $FuncSets$ partition the PUT's layout into segments, each representing a semantically similar relationship in functionality (e.g., address web forms, header navigation bar, social media component, footer links). Let the $FuncSets F_1, F_2, \dots, F_n$ of V be the n partitions, such that $F_1 \cup F_2 \cup \dots \cup F_n = V$ and $F_1 \cap F_2 \cap \dots \cap F_n = \emptyset$, meaning that an element in the PUT can only reside in one (and only one) $FuncSet$. Navigation that follows sequences and relationships within the PUT's linear logical content flow should enter and exit each $FuncSet$ exactly once. We define a failure (true detection) as the existence of (1) more than one incoming **TAB** edge or (2) more than one **SHIFT+TAB** edge that enters a $FuncSet V_j$. When there exists more than one way to navigate to a given $FuncSet$ from either navigation direction, it represents that the focus violates the sequential

navigation order of that partition. Such behavior means that the way a keyboard user encounters the elements in the partition does not follow the visual linear flow of the content within the partition.

In Figure 6's example UI, the "Email" input textbox is semantically clustered with its adjacent "Forgot email" link (corresponding to the cluster F_2 in the KFFG). The same applies to the "Password" input textbox and the "Forgot password" link that form their own cluster (the cluster F_3 in the KFFG). A Unintuitive-Nav-Order KNF is identified because two **TAB** edges and two **SHIFT+TAB** edges can navigate to these two clusters in the KFFG.

Please note that our approach defines the nodes for the KFFG as the set of visible elements. Screen reader links that are not initially visible (e.g., skip navigation links or parallel links that are hidden for the sole purpose of providing labels for other control elements) are not used in our semantic clustering, thus, they will not be classified as KNFs.

5.1.2 Unintuitive Change-of-Context. This KNF occurs when the keyboard navigation triggers an unexpected change to the context of the PUT that lacks a prompt for users to abort the action. To detect this, the approach first examines if the special node " v_{ext} " in the KFFGs has any incoming edge leading to it whose action ϕ is a non-actuation key (i.e., $\phi = \mathbf{TAB}, \mathbf{SHIFT+TAB}, \uparrow, \downarrow, \leftarrow, \rightarrow$). Such an edge indicates an attempt to unload (navigate away from) the PUT through keystrokes that are not intended for activation. Per standard web design practices, non-activation keys are generally not intended to execute an action to confirm a web dialog prompt.

In Figure 6's example UI, the language selection drop-down control contains an Unintuitive Change-of-Context KNF. The corresponding KFFG shows that three controls in the UI (i.e., the language selection drop-down, the "Help center" and the "Privacy" links) will cause a keyboard user to navigate away from the current page (as represented by the blue edges). In this example, only the language selection drop-down is identified as a KNF because it has an outgoing edge whose action ϕ is a non-activation action.

5.2 Detecting Unapparent Keyboard Focus

To detect **Unapparent Keyboard Focus** type KNFs, whose behaviors are described in Section 2.2, the goal is to identify those interactive elements in the PUT that lack sufficient visual change when they receive focus during keyboard navigation [47].

The approach follows the focus appearance metrics as stated in the working draft of WCAG 2.2 [50] to determine what is considered sufficient focus (1) for the users that have difficulty perceiving UI components' focus ring, and (2) for the users that have difficulties perceiving the difference in contrast from the focus ring. For a given UI element v , the definition of sufficient visible focus can be represented using the following definition:

$$v_{visible} := \{v \mid v \in V \wedge C(v) \wedge (\mathcal{M}_1(v) \vee \mathcal{M}_2(v)) \wedge \mathcal{A}(v) \wedge \mathcal{O}(v)\} \quad (1)$$

Here, the predicate $C(v)$ represents the condition that satisfies a **contrasting area**; the predicates $\mathcal{M}_1(v)$ and $\mathcal{M}_2(v)$ represent the two conditions for a **minimum area**; the predicate $\mathcal{A}(v)$ represents the condition that satisfies **adjacent contrast**; and the predicate $\mathcal{O}(v)$ represents the condition that satisfies **obscurity** of the element.

5.2.1 Contrasting Area. In order for an element's focus indicator to be considered visible, there must exist an area in the focus indicator that has a sufficient contrast ratio between the colors in the focused and unfocused states.

At a high level, detecting a contrasting area could be done by comparing the visual representation (screenshot) of every $v \in V$ before and after they receive focus. Related work [82–84, 86] has shown that differences in pixels can be used to reliably detect visual inconsistencies of HTML elements. Our approach identifies the visual differences between I and I' using perceptual image differencing (PID), a computer vision based technique for image comparison [96]. This visual difference is represented as the set of perceivable difference pixels DP_Δ between I and I' . The PID uses a Δ parameter as a customizable tolerance level to indicate how closely I and I' must match. We employed the parameters used in related research [85] used to address visual differences in web applications that constitute as presentation failures. If the PID determines that there is a sufficient human perceivable difference, this means there exists a set of pixels that are visually different between when the element is in focus versus not in focus (i.e., $DP_\Delta \neq \emptyset$). When such differences exist, the approach then analyzes the region of perceptual difference (i.e., DP_Δ) in the focused state (i.e., $DP \in I$) and unfocused state (i.e., $DP' \in I'$) to determine if the colors of the pixels in DP and DP' have sufficient contrast. To do this, for every pixel in DP , we find its corresponding pixel in DP' and compare the color value of these two pixels using the color contrast formula defined by WCAG [38].

Let $x = (x^r, x^g, x^b)$ and $y = (y^r, y^g, y^b)$ be a pair of dominant colors represented in their RGB colorspace, where $x \in DP$ and $y \in DP'$. The contrast ratio (CR) between them is mathematically defined by the relative luminance of the lighter color over the relative luminance of the darker color, as shown in Equation 2a, where $L(x)$ is the relative luminance defined in Equation 2b and Equation 2c.

$$CR(x, y) = \frac{\max(L(x), L(y)) + 0.05}{\min(L(x), L(y)) + 0.05} \in [1, 21] \quad (2a)$$

$$L(x) = 0.2126 \cdot h(x_r) + 0.7152 \cdot h(x_g) + 0.0722 \cdot h(x_b) \quad (2b)$$

$$h(x) = \begin{cases} \frac{a/255}{12.92} & \text{if } a/255 \leq 0.03928 \\ \left(\frac{a/255 + 0.055}{1.055} \right)^{2.4} & \text{otherwise} \end{cases} \quad (2c)$$

If there exists a pair of corresponding pixels that results in a Contrast Ratio (CR) of at least 3:1, then the contrast area CA is represented by the set of those pixels that satisfy the CR, otherwise, $CA = \emptyset$. The predicate $C(v)$ yields true if CA is not an empty set ($CA \neq \emptyset$), otherwise, false.

5.2.2 Minimum Area. If there exists a **contrasting area** (i.e., $CA \neq \emptyset$), then our approach uses the two conditions to check whether the region covered by the set of pixels CA satisfies the threshold of a **minimum area**. The first condition (i.e., predicate $\mathcal{M}_1(v)$) requires CA 's area to be at least as large as the area covered by the outline of a 1 CSS pixel (i.e., normatively defined as the absolute length px) thick perimeter of the element's MBR. The second condition (i.e., $\mathcal{M}_2(v)$)

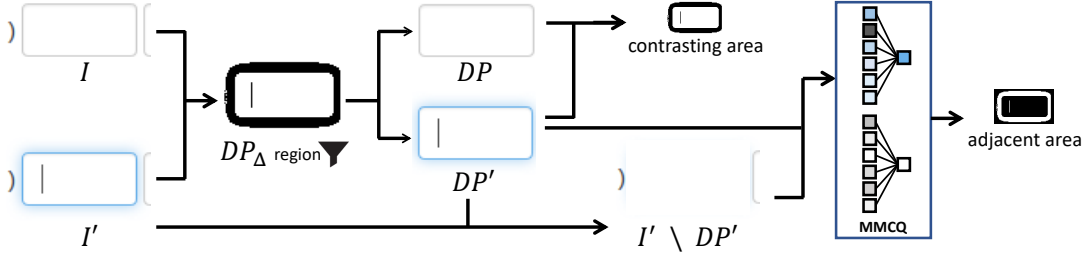


Figure 7: A flow diagram that shows the overall process of analyzing a textbox’s focus indicator from screenshots of its unfocused (I) and focused (I') states. The process yields the contrasting area from the area of visual changes between the focused (DP') and unfocused (DP) states; and the adjacent contrast from the focused area (DP') and adjacent areas ($I' \setminus DP'$).

requires CA ’s area to be at least as large as the area covered by a 4 CSS pixel thick line along the shortest side of the element’s MBR. In order to translate these minimum areas from web rendering metrics into actual screen pixels, we capture the equivalent (before and after) screenshots of a dummy element for each element-under-test. We create such dummy elements by overriding the original element-under-test’s focus events and CSS properties to portray a visible representation of these outlines, for each of the two conditions. We then use the difference in pixels of these dummy elements before and after they receive focus to calculate a pixel-based threshold for the minimum area for the original element-under-test. If $C(v)$ yields true, then the predicates $M_1(v)$ and $M_2(v)$ respectively represent the two true/false conditions for a **minimum area**.

5.2.3 Adjacent Contrast. In addition to having a contrasting area CA with sufficient contrast between the colors in the focused and unfocused states (i.e., DP' and DP), the element’s focus component is also required to have **sufficient contrast** with respect to its surrounding color(s). In other words, DP' must satisfy CR against those pixels adjacent to the focus indicator region (i.e., $I' \setminus DP'$) in the focused state.

Finding such an adjacent contrasting area is complicated by several challenges. The first challenge is that there is no corresponding 1:1 mapping to compare colors with when the pixels in DP' and $I' \setminus DP'$ do not overlap each other. The second challenge is that browsers render text or shapes using anti-aliasing to smooth the color transition of edges by introducing some intermediate colors around the edges. The third challenge is that there are various styles of focus indicators (i.e., dotted lines or changes to background colors) that may consist of multiple color-related properties. These variants make checking adjacent contrasts ambiguous and difficult to achieve. To mitigate the ambiguity, our approach utilizes a color quantization method used in CV called Median Cut Quantization (MCQ) [97] to extract the dominant color(s) to reduce color noise. The intuition is that the visual difference region can be perceived as a single or discrete set of colors based on human perception. Instead of simply taking the average of the colors in DP' and $I' \setminus DP'$, which may lead to an incorrect representation of colors, our approach uses Modified-MCQ (MMCQ) [7] to identify the dominating colors for a given image region using dimensionality reduction that decreases the number of colors used to find the $n = 2$ dominant colors in both DP' and $I' \setminus DP'$. If there exists a pair of corresponding dominant colors in the set of all ordered pairs $(x, y) \in (DP' \times I' \setminus DP')$, that

yields a minimum CR of 3:1, then this predicate $\mathcal{A}(v)$ yields true, otherwise, $\mathcal{A}(v)$ yields false.

5.2.4 Obscurity. The approach also checks for whether the element with focus is not **fully obscured** or hidden by another page content [40]. To do this, we came up with heuristics to analyze the rendering of the web page. The heuristic first checks that the element itself is not a descendent of another element that is hidden with properties `type="hidden"`, `visibility:hidden`, or `display:none`. Next, the heuristic checks that the element is not obscured by another element. This is done by ensuring that the element does not collide with any other element’s MBR where the other element is rendered above, according to “painting orders” defined by the CSS Positioned Layout [1] (i.e., the browser’s rendering engine). All of these heuristics can be identified by analyzing the attributes of the elements in the DOM of the PUT when it has been rendered in the browser. The predicate $O(v)$ yields true if the element is partially obscured or hidden, otherwise, $O(v)$ yields false. Note that our approach can be customized to handle full obscurity [41]. This is done by checking if the element’s MBR (i.e., R) is fully contained by another element. Let R_{e_1} and R_{e_2} be the MBRs of elements e_1 and e_2 , e_1 is obscured (contained) by e_2 if $R_{e_1}.x_1 \leq R_{e_2}.x_1 \wedge R_{e_1}.y_1 \leq R_{e_2}.y_1 \wedge R_{e_1}.x_2 \geq R_{e_2}.x_2 \wedge R_{e_1}.y_2 \geq R_{e_2}.y_2$.

Figure 7 demonstrates the process of analyzing the keyboard focus for an input form UI element. On the left, the element’s unfocused (i.e., I) and focused (i.e., I') screenshots are compared and used to determine the visually perceivable difference DP_Δ via PID. DP_Δ is then used as a filter to determine the area of focus in I and I' , which are respectively denoted as DP and DP' . The corresponding pixels in DP and DP' are then compared for color contrast to determine whether a contrasting area exists or not. Sufficient adjacent contrast is determined in a similar way by comparing the dominant colors in DP' versus $I' \setminus DP'$ using MMCQ.

6 EVALUATION

To assess the effectiveness of our approach, we conducted an empirical evaluation that focused on two research questions:

RQ1: What is the accuracy of our technique in detecting KNFs in comparison with state-of-the-art approaches?

RQ2: How fast is our technique in detecting KNFs?

The detailed data of our evaluation is made available to the community via the project website [51].

6.1 Implementation

We implemented our approach as a Java-based prototype tool, called **Keyboard Navigation Failure Locator (BAGEL)**. The approach used Selenium WebDriver version 3.141.5, an automated browser testing tool to load, render, and interact with the subject web pages to build the KFFG model. In particular, the FirefoxDriver API [27] is used to send keyboard actions to the page and to execute JavaScript-based code to capture the change to the browser’s keyboard focus. All web pages were rendered with a fixed screen resolution of 1920×1080 pixels on Firefox version 92.0. The approach also used iFix’s [81] DBScan clustering to help identify the different *FuncSets* in a web page. We used aShot, a screenshot-capturing utility to capture a complete representation of the browser’s viewport. We ran BAGEL and all experiments on a single AMD Ryzen 7 2700X 64-bit machine with 64GB memory and Ubuntu Linux 18.04.4 LTS.

Note that the setting that we used in our experiment to analyze Unapparent-Focus uses the minimum contrast ratio of 3.1 as defined in SC 2.4.11 *Focus Appearance* [39] and the partial obscurity as defined in SC 2.4.12 *Focus Not Obscured* [40], which are the minimum required levels of compliance, meaning they are the most critical thresholds to ensure websites meet the most essential needs of accessibility. Our approach is fully customizable to handle higher contrast ratios (e.g., 4.5:1 or 7:1) or full obscurity [41] for higher levels (e.g., AA or AAA) of WCAG compliance.

6.2 Subject Web Pages / Accessibility Tools

We conducted our evaluation on a set of 20 real-world subject web pages gathered from two sources. The first source is the Moz 500 top websites list [35] and the second source is a list of randomly selected websites that offer information, products, and services. We chose these two sources because together, they include government, education, as well as company websites that are obligated for accessibility as mandated by the ADA’s Title II and Title III regulations [21, 58]. To come up with the list of websites for the second source, we first used Google search to look for popular online government, university, community forums, e-commerce, and service websites. We searched the keywords “*list of [genre] websites*” for the five genres and included those that were listed under the “featured snippets” [24] of each search result (e.g., Science.gov, Harvard.edu, Twitch, eBay, Domino’s Pizza). From these results, we compiled an initial list of 25 websites. To ensure our population was also representative of less popular sites, we extended this initial list using *similarsites.com* to randomly select two alternative websites of the same genre for each of the listed websites. Together, the second source consists of 75 unique websites. For the selection protocol, we randomly selected web pages from these two sources and manually interacted with them (following the WCAG Techniques used to identify KNFs) to include those that contained at least one type of KNF. The process was repeated until we had 10 subjects from each source. Note that our final set of 20 subjects did not include web pages from government and education websites because we did not observe any KNFs in these types of web pages that we encountered.

A challenge for our evaluation was that since the subject web pages represented popular and well-maintained websites, they were under constant evolution. To provide consistency in our results, we captured a complete version of each subject web page at the time we added it to the subject pool. To do this, we used mitmproxy [12] to store all the page’s resources coming through HTTP/HTTPS traffic, including the JavaScript that is responsible for the web pages to remain interactive. The captured HTML, CSS, JavaScript, and other binary resources are packaged as local files to be loaded and replayed back by the proxy. For the few subjects that were not able to properly cache due to extra runtime security validation, we used web scraping tools [9], to capture the rendered resources directly from the DOM. Such a method is necessary for the subjects to behave the same as intended so our experiments could be repeatable.

To compare the effectiveness of BAGEL against current accessibility approaches, we selected available state-of-the-art tools from the *Web Accessibility Evaluation Tools List* [5] provided by W3C. Since there are no known tools that specifically target KNFs, we included prominent industry tools *WAVE* [18], *ARC Toolkit* [31], *Axe DevTools Pro* [23], *Tenon Check* [36] as well as a tool from the research literature *QualWeb* [65]. This set of tools offers the most complete analyses in handling a wide spectrum of issues that leverage analyses on the rendered DOM after scripting and CSS styling are applied [19]. We included these tools to see the helpfulness and extent their output can relate to KNFs.

6.3 Procedures

To answer RQ1 we ran BAGEL and measured how accurately it detected KNFs in the subject web pages. The accuracy was measured in terms of precision and recall for each of the three types of KNFs (Unintuitive-Nav-Order, Unintuitive Change-Of-Context, and Unapparent-Focus). To find the KNFs in each subject web page, the authors manually interacted with the page’s UI and their underlying functional components via the keyboard to create the ground-truth. The process followed the Success Criterion defined by WCAG for each KNF type, including the testing techniques that point out objectively, the exact behaviors of the failures that a keyboard-based user would experience.

We analyzed BAGEL’s output to check if it identified the actual element responsible for each KNF with respect to the ground-truth. We calculated false-positives, true-negatives, and false-negatives in an analogous way. For Unintuitive-Nav-Order, we included as the faulty elements, all of the elements contained in those clusters (*FuncSet*) where the navigation enters from more than one entry point. For Unintuitive Change-Of-Context, we included those elements that caused the web page to navigate away when some non-activation keyboard actions were performed on them. For Unapparent-Focus, we included those elements that did not have sufficient visible focus indication when receiving focus. These ways that we identify faulty elements follow the same workload pattern [52] that a developer would use the output of BAGEL (an outputted list of faulty elements) to debug the problems on a web page.

To measure how well BAGEL performed against other accessibility scanners, we ran the five state-of-the-art tools and measured how accurately they detected KNFs in the subject web pages. Since

Table 1: Results of KNF detection accuracy for all tools (RQ1) and their average run-time per subject (RQ2).

Tool	Unintuitive-Nav-Order		Change-Of-Context		Unapparent-Focus		Run-time (min:sec)
	Precision	Recall	Precision	Recall	Precision	Recall	
BAGEL	85%	100%	83%	83%	97%	92%	07:32
Axe DevTools Pro	49%	42%	0%	0%	0%	0%	00:10
QualWeb	57%	80%	0%	0%	0%	0%	00:23
WAVE	60%	47%	100%	50%	0%	0%	00:03
Tenon Check	0%	0%	0%	0%	0%	0%	00:19
ARC Toolkit	50%	42%	0%	0%	0%	0%	00:05

Table 2: Results of BAGEL’s KNF detection accuracy (RQ1) and run-time (RQ2) detailed for each subject web page. The “n/a” represents that the precision/recall does not apply (usually due to the web page not including the specific type of KNF).

Subject	Unintuitive-Nav-Order		Change-Of-Context		Unapparent-Focus		Run-time (min:sec)
	Precision	Recall	Precision	Recall	Precision	Recall	
alibaba	100%	100%	0%	0%	100%	100%	11:12
arxiv	100%	100%	n/a	n/a	100%	94%	04:21
dickssportinggoods	67%	100%	n/a	n/a	100%	75%	03:52
disneyworld	100%	100%	n/a	n/a	94%	94%	05:56
engadget	n/a	n/a	n/a	n/a	95%	100%	08:12
experian	100%	100%	n/a	n/a	100%	100%	03:18
github	100%	100%	n/a	n/a	100%	100%	03:02
leagueoflegends	n/a	n/a	100%	100%	100%	80%	03:23
lenovo	n/a	n/a	100%	100%	100%	86%	39:08
mozilla	n/a	n/a	100%	100%	100%	77%	12:26
papajohns	0%	n/a	100%	100%	100%	100%	11:22
researchgate	100%	100%	n/a	n/a	94%	100%	03:49
robinhood	n/a	n/a	n/a	n/a	100%	100%	01:53
samsclub	86%	100%	n/a	n/a	67%	100%	03:40
theguardian	n/a	n/a	n/a	n/a	100%	91%	03:56
ticketon	n/a	n/a	n/a	n/a	100%	80%	04:19
trelo	n/a	n/a	100%	100%	100%	100%	06:54
woot	100%	100%	n/a	n/a	89%	89%	03:10
wsj	n/a	n/a	n/a	n/a	98%	100%	10:11
yelp	n/a	n/a	n/a	n/a	100%	81%	06:34

*The URLs are for reference only. Experiments were conducted on the cached version of the subject web pages.

each tool had its own way of reporting detection, to make a comparison across the different tools, we objectively interpreted their outputted results based on violations to specific WCAG Success Criterion (SC). For these five tools, we considered reports with any mention of keyboard accessibility issues related to WCAG SC 2.4.3, SC 2.4.7, SC 3.2.1, and SC 3.2.2 as a KNF detection for the corresponding subject web page. We do this because these Success Criteria are the exact guidelines in which the KNFs were defined. For all five tools, our mechanism for recognizing a detection at the *per-element* level was consistent with the tool’s intended usage and resulted in the most favorable accuracy scores for the tool.

To answer RQ2, we measured the time it took to run the BAGEL tools on each subject web page. The running time included the time to start the tool, load the browser, build the models, and compute the detection results. For *WAVE*, *ARC Toolkit*, and *Axe DevTools Pro*, which are browser extensions, we included the time from when the detection was executed until the results were displayed. For *Tenon Check* and *QualWeb*, which are web-based services, we included

the time to submit a subject link for analysis until the result page was displayed.

6.4 Discussion of RQ1 Results

6.4.1 Detection accuracy of BAGEL and other tools. The results of our experiments are shown in Table 1 and Table 2, which respectively represent the KNF detection accuracy for BAGEL compared to other tools and the detailed accuracy for individual subjects. Generally, BAGEL was able to detect KNFs with higher accuracy compared to the other tools. None of the other tools are capable of detecting all three types of KNFs.

Table 1 shows that BAGEL was able to accurately detect the faulty element(s) responsible for each of the three types of KNFs with high precision and recall (with an F1 score of 93%, 83%, and 95% for the three types of KNFs). *Tenon Check* cannot detect any of the KNFs at all. Although *WAVE* had a higher precision than BAGEL in detecting Change-Of-Context KNFs, it failed to detect many and therefore had a low recall. Overall, the other tools (*Axe DevTools*



Figure 8: An example of false-positive Unintuitive-Nav-Order detection in the subject *papajohns*. The misclassification of the red (dotted line) and green (solid line) clusters caused the focus to appear to move between these two clusters

Pro, *QualWeb*, *WAVE*, and *ARC Toolkit*) have only an average F1 score (across the three types of KNFs) of 15%, 23%, 43%, and 15% respectively. This shows that BAGEL does a much better job in detecting elements that are responsible for KNFs in the subject web pages. Table 2 shows that BAGEL was able to detect all the Unintuitive-Nav-Order KNFs in the subjects, all the Change-Of-Context KNFs in five out of six subjects that had this type of KNFs, and correctly detect all Unapparent-Focus KNFs in 14 out of 20 subjects that had this type of KNF.

6.4.2 Insights on What the Other Tools Detect. *WAVE*, *Axe DevTools Pro*, and *ARC Toolkit* detect elements with Unintuitive-Nav-Order KNFs by scanning the DOM of a web page for elements with a `tabindex` value larger than zero. These elements are then outputted as potential issues. While this may be a simple proxy to detect potential order issues, a positive `tabindex` value does not always cause problems for page navigation. Therefore, these tools have higher rates of false-positives.

All of the elements responsible for Change-Of-Context KNFs correctly identified by *WAVE* were menu-items binded with JavaScript onchange event handler that triggered a new page when an element in a menu was selected. *WAVE* was able to identify these KNFs by scanning the web page’s DOM and identifying any menu with such an attached event handler. However, *WAVE* was not able to identify other cases of Change-Of-Context where an element needs to be dynamically interacted with to trigger this faulty behavior. This under-approximation caused *WAVE* to miss many such KNFs. For *QualWeb*, it considers any web form without a submit button to be a potential issue that impacts WCAG SC 3.2.2 and thus, a true detection. However, using this metric, *QualWeb* was not able to correctly identify any element with Change-Of-Context KNF.

In terms of Unapparent-Focus, *QualWeb* relies on scanning the DOM and the CSS applied to a web page to identify elements that do not have visible focus indications. However, since this is not a conclusive way of detecting Unapparent-Focus KNFs (as discussed in Section 2.2), *QualWeb* displays these as a potential warning that needs to be manually examined and verified by the users. While this can be very helpful, it can lead developers to miss many instances of Unapparent-Focus and requires them to examine many false positives. In fact, *QualWeb* was not able to detect 101 Unapparent-Focus KNFs, while 330 of the elements that *QualWeb* detected were false-positives.

6.4.3 Discussion of BAGEL’s Negative Results. We investigated the KNFs that our approach incorrectly detected and found several scenarios where this occurred.

For Unintuitive-Nav-Order, the false detection was generally due to the inaccurate clustering of our *FuncSets*. For example, in the subject *papajohns* as shown in Figure 8, the footer links of the page are presented in four categories “Our Company”, “Our Pizza”, “Help”, and “Career”. The similarity of these category headers was higher because they are made of `<button>` elements, thus, they were separately grouped together (into the dotted cluster) from the rest of the links in the footer. As a result, the keyboard navigation crossed between these two groupings multiple times (i.e., in the forward direction 1 → 2; 4 → 5 → 6, etc.), which caused a false-positive detection because we do not consider this navigation as a Unintuitive-Nav-Order KNF.

For Unapparent-Focus, the inaccurate detections were usually due to screenshots not being correctly captured between the non-focused and focused states. In some cases, the screenshots resulted as empty white images or included black portions in the focused state, resulting in the perceivable difference and the contrast being falsely identified to be sufficient (false-negatives). In the subject *arxiv*, some extraneous pixels from other elements were included as part of the screenshot in the focused state. This caused the dominant color in the outer/adjacent region to be falsely identified, resulting insufficient contrast (false-positives). Another reason that resulted in false-negative detection (in subject *leagueoflegends* and *lenovo*) was because the elements that lacked visible focus were also not accessible to the keyboard. Specifically, these elements were implemented using customized `<div>` and `<h2>` that could not receive keyboard focus. As a result, Selenium WebDriver was unable to set these into focus to capture their screenshots. We also found examples in *woot* where the dummy elements that we used to capture the minimum area baseline could not be correctly created because the CSS could not be overwritten. This resulted in the minimum area threshold being set to zero and caused a false-negative detection.

6.5 Discussion of RQ2 Results

The run-time of BAGEL was significantly slower than the other five approaches. It took BAGEL an average of over 7.5 minutes to complete the detection for each subject. We analyzed the run-time breakdown of each individual step in BAGEL in detail and found that approximately 98% of the total time was spent on modeling

and building the KFFG and only 2% was spent on the detection. As shown in Figure 9, the KFFG construction process itself spent 59% of the time building the visual representation (screenshots) for the elements, 14% of the time building the keyboard navigation edges, and <1% of time performing the clustering on the elements. The average time for detecting all three KNFs was around 9 seconds (2% of total time) from the already constructed KFFGs.

BAGEL took much more time because it dynamically interacts with the subjects in the same way a keyboard user would. It also takes screenshots to capture what a sighted user actually sees during their interaction. The higher time cost is offset by its higher precision and recall. As an unoptimized prototype, we think that ~8 minutes is a reasonable time in the context of software engineering (e.g., as compared to unit and integration testing tasks) and the current run-time can be improved. Strategies to reduce the run-time include deploying and distributing Selenium’s processing across multiple cloud computing instances. Since the majority of the run-time overhead lies in extracting the web elements and capturing the screenshots, we expect that future improvements to Selenium as well as alternative capturing techniques would also improve BAGEL’s run-time.

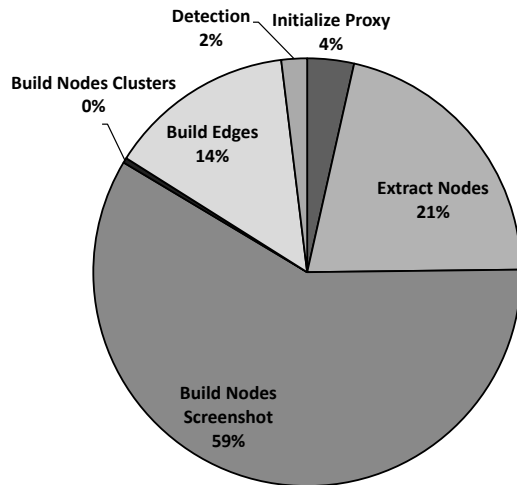


Figure 9: Run-time breakdown of each individual step in our approach.

6.6 Reflection on the Root Causes of KNFs

We analyzed the subject web pages and found several implementation choices that contributed to the occurrence of the detected KNFs. Here, we look at the common root causes that we discovered from our study. This outcome can be useful for the developers to prevent these implementation mistakes from happening in the future.

The Unintuitive-Nav-Order type KNFs were usually caused by the developers explicitly assigning `tabindex` values to input controls. In our subjects, we found many instances where customized `tabindex` attributes were used to override the default tab order in order to automatically set the initial focus into input text-boxes

where the user enters info (e.g., user id on a login page). Such implementations happened frequently on log-in forms, which is a common practice to seamlessly allow users to begin entering login credentials upon page load without extra navigation.

In terms of Unintuitive Change-Of-Context type KNFs, we discovered that they usually occurred when users change selections in custom drop-down widgets. These custom widgets often have their own scripting to trigger the unexpected Change-Of-Context. We found that the default Google Translate Widget [33] is also prone to this KNF despite it being implemented with a native `<select>` component. The scripting of the widget causes the web page to change as soon as users change the preferred language from its drop-down menu to automatically translate the web page.

For Unapparent-Focus type KNFs, we found that they frequently occurred under two scenarios. The first scenario was that there was no focus indication at all. This occurred when developers explicitly set the outline CSS property to `outline: 0`, `outline: none` or adopt custom CSS libraries or CSS Reset [8] to disable the browser’s default focus outline. Upon investigation, we found this is a common practice among developers to ensure cross-browser aesthetics consistency. To maintain aesthetics, developers also override and hide the default outline with JavaScript to prevent the focused element from being highlighted when it receives programmatic focus from a mouse. Such practice may cause side effects that suppress the focus ring when certain screen readers fire mouse events to the browser from keyboard interactions (e.g., user presses **ENTER** and the AT programmatically fires a `mousedown` event [2]). The second scenario is for a focus indicator to exist but then fail to meet the threshold of a minimum contrasting area. In such cases, the elements often do not have an explicit focus style applied to them, therefore resulting in a faint, dotted, black line supplied as Firefox’s default focus indicator. This focus ring often does not itself pass the 1 CSS pixel thick perimeter outline minimum area threshold. However, when the element’s MBR is large enough, the default focus indicator does have enough pixels to pass the second threshold criterion of a 4 CSS pixel thick line along the MBR’s shortest side. Therefore smaller elements with default focus indicators tend to not have sufficient pixels to pass the minimum contrasting area.

6.7 Threat to Validity

A potential threat to construct validity is that our definition of KNFs may not match the intended idea of actual accessibility issues in the real-world setting. However, the use of this definition is reasonable because it is based on success criteria defined by WCAG, which is designed by accessibility experts from around the world and is used as a basis to determine websites’ compliance with ADA law [34]. As further validation, we conducted an empirical study of KNFs to qualitatively show the impact and severity of our detected KNFs from the two complementary perspectives of developers and users.

6.7.1 Feedback From Developers’ Perspective. We reached out to the developers of our subject web pages (via the websites’ contact forms, email, or their web app repository on GitHub) to report the discovered issues for feedback. Overall, out of the 12 subjects that we were able to reach out to, five developers acknowledged receiving our report, and two of them personally expressed the importance of our findings. We also received positive feedback

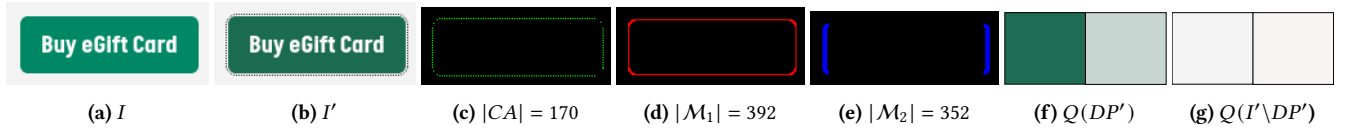


Figure 10: An example of a “Buy eGift Card” button in the subject *papajohns* that changes color before (I) and after (I') receiving focus. The example demonstrates the effective contrast area $|CA|$ between I and I' is 170, which is less than both the minimum area thresholds of $|M_1| = 392$ and $|M_2| = 352$ as suggested by WCAG. Therefore, this element has an Unapparent-Focus type KNF

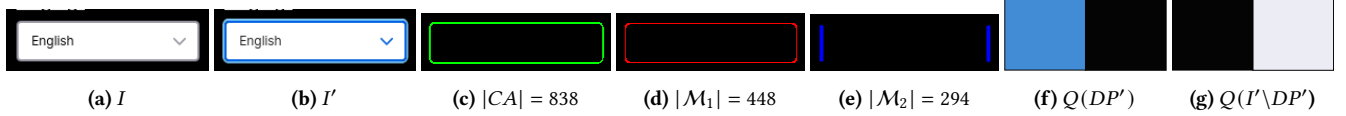


Figure 11: An example of a language drop-down menu in the subject *mozilla* before (I) and after (I') receiving focus. In this case, the effective contrast area $|CA|$ between I and I' is 838, which passes both the minimum area thresholds of $|M_1| = 448$ and $|M_2| = 294$. The adjacent contrasts between the colors in the focus area $Q(DP')$ and the colors in the area outside the focus area $Q(I' \setminus DP')$ are also above the minimum of 3:1 as defined by WCAG. This element does not have an Unapparent-Focus type KNF.

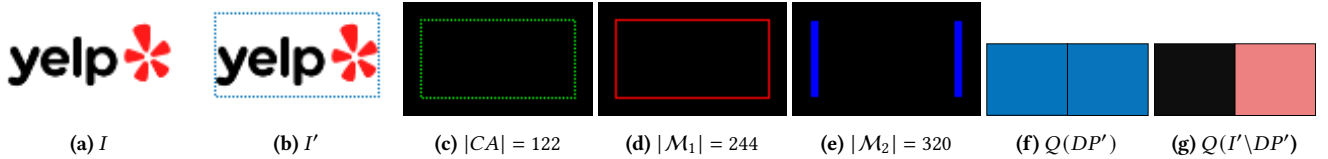


Figure 12: An example of a link in the subject *yelp* that uses a blue version of Firefox’s default focus indicator style before (I) and after (I') receiving focus. The effective contrast area $|CA|$ between I and I' is 122, which is less than both the minimum area thresholds of $|M_1| = 244$ and $|M_2| = 320$ as suggested by WCAG. Therefore, this element has an Unapparent-Focus type KNF.

from developers stating that they were addressing the issues or that our report could help improve the accessibility of their websites. After discussing the details of the reported KNF and confirming the replication of the issues, the Product Lead for *Engadget* stated that they will get the KNFs into the queue to promptly fix them. According to *arxiv*’s Content Management team, repairing these issues (KNFs) “*is extremely important to our mission*”. In addition, we found that three of the subjects were partially fixed since we first reported the issues. Together, this represented positive validations for nearly half of the subjects. We believe that the positive responses that we received, alongside our reliance on the WCAG success criteria, show that the issues we are targeting represent real-world problems.

6.7.2 Feedback From Users’ Perspective. We included a small user study to gather feedback from two keyboard users with disabilities. The first user (U1) is a blind user that navigates the web via a screen reader and the second user (U2) is a sighted user with a motor disability that uses the keyboard instead of a mouse. We randomly selected six subject web pages (denoted as S1 to S6) and gave U1 and U2 five minutes to interact with each of the web page’s core functionalities. We then discussed whether they found any difficulties with the navigation and how they felt the usability can be improved on this web page to make it more accessible. The result

shows that, in general, both U1 and U2 did not find the web pages easy to navigate.

For Unintuitive-Nav-Order KNFs, based on U1’s interaction with S1 via a screen reader, he believed there was an issue with the flow, making it “*not easy to understand at first*”. However, he does not consider the navigation a major issue because the page size of S1 is relatively small, thus it was “*not that difficult to manage*”. When it comes S2, U1 believed the scattered information is more severe because he had to navigate back and forth more times to fully understand the content when the page is larger and more complex. He mentioned, “*there’s no way to jump around, especially when there’s a toolbar*”.

For Change-Of-Context KNFs, U1 did not encounter the KNF on S3 because he was using the VoiceOver screen reader on an iPhone, which displayed an iOS-based dialog to override the behavior of the faulty drop-down implementation. However, after informing U1 about the KNF, he expressed that the particular issue occurred to him often when browsing the web on a computer – especially noticeable in banking sites. U1 expressed that “*It’s frustrating for me because when you go down and it starts loading. Usually, I don’t understand what happened until like later. Uh, maybe I did something wrong or something. You think that you made a mistake*”. From U2’s sighted user perspective, he indicated the same issue with S4, that the web page “*ended up redirecting me to a page that I did not want*”.

For Unapparent-Focus KNFs, U2 expressed that during his interaction with S5, *“It was difficult to see the highlighted areas. especially when I got to the end of the page, it was difficult to follow where the focus jumped back in the page.”* He further emphasized that *“It slowed me down. I also needed to spend time trying to find the highlighted item”*. In terms of S6, where the keyboard focus indicator is completely absent visually, U2 mentioned *“I did not know what was being selected on the web page. I was trying to select one of the options but not knowing where my cursor was, it was harder to know if I was selecting the correct option”*.

In terms of improvements, U1 and U2 generally expressed the need to circumvent the failure of each KNF, including *“making the page flow more direct and to the point”* for Unintuitive-Nav-Order; *“having a preview menu as well as being able to highlight the option without selecting it”* for Change-Of-Context; and *“having a noticeable, bolder highlighted areas around the selected part of the page”* for Unapparent-Focus. These responses verified that what BAGEL identifies as KNFs aligns with issues faced by real-life users with disabilities that rely on keyboard navigation.

7 LIMITATIONS AND FUTURE WORK

7.1 Approach Limitations

A primary limitation of our approach is that it is only able to explore and identify KNFs in a single web page. In addition, within the single web page, the approach is only able to identify those KNFs that exist in the web page’s initial UI state. For example, in modern Single-page applications, users can interact with a web page to trigger the appearance of different web components (e.g., hidden `<div>` panel or modal dialogs). If a faulty element is initially hidden when the page loads, only to become visible deeper down the navigation, our approach will not be able to analyze them. By integrating our KFFG modeling with a reliable crawling mechanism, we believe the approach can more completely explore web pages in the future.

7.2 Evaluation Study Limitations

A limitation of our evaluation is that our subjects are only a single web page within each of the selected websites. However, our sample selection process followed the Website Accessibility Conformance Evaluation Methodology (WCAG-EM) [17] to include web pages that cover (1) essential functionality of the website, such as the Log-In and Sign-Up page, (2) common and relevant key web pages, such as the Contact Us and Support page, and (3) a variety of content using different web technologies, such as Bootstrap, AngularJS, and WordPress plugins. The subjects cover different Rich Internet Application (RIA) technologies comprising modern HTML5 and custom JavaScript widgets, such as sliders, drop-down menus, and navigation menu components. Future work could broaden our findings by studying a broader set of subject web pages to identify more design and implementation mistakes that developers can make to cause KNFs.

Another limitation of our evaluation is that our tool is implemented and evaluated using Selenium’s FirefoxDriver and there could be potential discrepancies between how different browsers show their default focus ring. For example, the styling of the default outline can vary by browser, where Edge and Firefox use a dotted line while Chrome and Opera use a blue focus outline. Our

experiments are based on Firefox to create a lower bond baseline because Firefox has the least visible default focus compared to other browsers. It is important to note that these browser-specific KNFs could be easily detected by systematically running BAGEL using different Selenium WebDrivers for different browsers. As future work, we plan to expand the research to these implementations to identify browser-specific Unapparent-Focus KNFs and how a web page’s keyboard focus accessibility differs across different browsers.

8 CONCLUDING REMARKS

Keyboard navigation accessibility barriers can hinder keyboard-based users’ ability to effectively and predictably interact with a web page’s UI. In this paper, we presented a novel automated approach to detect navigation-based accessibility barriers related to focus navigation that are defined in WCAG, which we refer to as Keyboard Navigation Failures (KNFs). The approach builds a mathematical graph-based model to (1) capture the web navigation behaviors and (2) the focus appearance from a keyboard user’s perspective to effectively detect KNFs. The empirical evaluation showed that our approach was able to successfully detect KNFs in a set of real-world subjects. The results, overall, are very positive and indicate that our approach can help developers detect keyboard navigation issues in their web applications. This work can potentially help improve WCAG conformance and can serve as a baseline for future automated testing to benefit people with various types of disabilities that depend on keyboard input.

ACKNOWLEDGMENTS

We gratefully acknowledge the voluntary efforts of the two users with disabilities that participated in our user study and shared their personal experiences on web accessibility. We also gratefully acknowledge Dr. Michael Crabb, who served as a mentor for our paper in a previous submission and whose feedback and guidance helped us to significantly improve the paper.

This work was supported by the National Science Foundation under grant 2009045.

REFERENCES

- [1] 2015. CSS Positioned Layout Module Level 3: Detailed stacking context – Painting order. <https://www.w3.org/TR/2015/WD-css3-positioning-20150203/#painting-order>. Updated: 2015-02-03.
- [2] 2018. Deque: Accessible Focus Indicators: Something to :focus on. <https://www.deque.com/blog/accessible-focus-indicators/>. Updated: 2018-05-31.
- [3] 2019. Google Developers - Web Fundamentals: Introduction to Focus. <https://developers.google.com/web/fundamentals/accessibility/focus>. Updated: 2019-09-03.
- [4] 2019. MDN Web Docs: CSS Grid Layout and Accessibility. https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Grid_Layout/CSS_Grid_Layout_and_Accessibility. Updated: 2019-05-08.
- [5] 2019. Web Accessibility Evaluation Tools List. <https://www.w3.org/WAI/ER/tools/>.
- [6] 2020. Almost half of people with disabilities don’t use the Internet: but why? | Oxford Internet Surveys - OxiS. <https://oxis.oii.ox.ac.uk/blog/almost-half-people-disabilities-dont-use-internet-why/>
- [7] 2020. Color quantization using modified median cut by Dan S. Bloomb. <http://www.leptonica.org/>. Updated: 2020-07-28.
- [8] 2020. cssreset: What Is A CSS Reset? <https://cssreset.com/what-is-a-css-reset/>. Updated: 2020-11-04.
- [9] 2020. Github: WebScrapBook. <https://github.com/danny0838/webscrapbook>. Accessed: 2020-08-16.
- [10] 2020. MDN Web Docs: Keyboard-navigable JavaScript widgets. https://developer.mozilla.org/en-US/docs/Web/Accessibility/Keyboard-navigable_JavaScript_widgets. Updated: 2020-11-15.

- [11] 2020. MDN Web Docs: Ordering Flex Items. https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Flexible_Box_Layout/Ordering_Flex_Items. Updated: 2020-10-16.
- [12] 2020. mitmproxy: a free and open source interactive HTTPS proxy. <https://mitmproxy.org/>. Accessed: 2020-08-16.
- [13] 2020. Techniques for WCAG 2.0 – Placing the interactive elements in an order that follows sequences and relationships within the content. <https://www.w3.org/TR/2016/NOTE-WCAG20-TECHS-20161007/G59>. Updated: 2020-11-14.
- [14] 2020. Usability.gov: User Interface Elements. <https://www.usability.gov/how-to-and-tools/methods/user-interface-elements.html>. Updated: 2020-11-09.
- [15] 2020. W3C: Understanding Success Criterion 1.4.11: Non-text Contrast. <https://www.w3.org/WAI/WCAG21/Understanding/non-text-contrast>. Updated: 2020-10-02.
- [16] 2020. W3C: WAI-ARIA Authoring Practices 1.1 – Keyboard Interaction. <https://www.w3.org/TR/wai-aria-practices-1.1/#keyboard-interaction>. Accessed: 2020-08-28.
- [17] 2020. W3C: Website Accessibility Conformance Evaluation Methodology (WCAG-EM) 1.0. <https://www.w3.org/TR/WCAG-EM/#req3a>. Accessed: 2020-08-25.
- [18] 2020. WAVE Web Accessibility Evaluation Tool. <https://wave.webaim.org/>. Accessed: 2020-08-25.
- [19] 2020. Web Accessibility Testing Tools: Who tests the DOM? <https://karlroves.com/2013/09/06/web-accessibility-testing-tools-who-tests-the-dom>. Accessed: 2020-07-16.
- [20] 2020. World Health Organization: World Report on Disability. https://www.who.int/disabilities/world_report/2011/report/en/. Accessed: 2020-07-02.
- [21] 2021. ADA.gov: Guidance on Web Accessibility and the ADA. <https://www.ada.gov/resources/web-guidance/>. Updated: 2021-11-10.
- [22] 2021. CSUN Universal Design Center: Web Accessibility Criteria - Tab Order. <https://www.csun.edu/universal-design-center/web-accessibility-criteria-tab-order>
- [23] 2021. Deque: axe DevTools - The Accessibility Testing Toolkit For Developers. <https://www.deque.com/axe-devtools-accessibility-testing/>. Updated: 2021-11-23.
- [24] 2021. Google Search Help: How Google's featured snippets work. <https://support.google.com/websearch/answer/9351707>. Updated: 2021-11-10.
- [25] 2021. Nielsen Norman Group: Keyboard-Only Navigation for Improved Accessibility. <https://www.nngroup.com/articles/keyboard-accessibility/>
- [26] 2021. Techniques G107: Using "activate" rather than "focus" as a trigger for changes of context. <https://www.w3.org/WAI/WCAG21/Techniques/general/G107>. Updated: 2021-09-12.
- [27] 2021. The Selenium Browser Automation Project: WebDriver - Keyboard. <https://www.selenium.dev/documentation/en/webdriver/keyboard/>. Accessed: 2021-01-14.
- [28] 2021. Yale University Usability & Web Accessibility: Focus & Keyboard Operability. <https://usability.yale.edu/web-accessibility/articles/focus-keyboard-operability>. Accessed: 2021-06-19.
- [29] 2022. Access Assistant. <https://www.webaccessibility.com/>
- [30] 2022. Accessibility Insights for Web - Microsoft. <https://accessibilityinsights.io/>
- [31] 2022. ARC Toolkit. <https://www.tpgi.com/arc-platform/arc-toolkit/>
- [32] 2022. Cucumber: Tools & techniques that elevate teams to greatness. <https://cucumber.io/>. Updated: 2022-09-13.
- [33] 2022. Google Search Central: Google Translate's Website Translator - available for non-commercial use. <https://developers.google.com/search/blog/2020/05/google-translates-website-translator>. Accessed: 2022-04-11.
- [34] 2022. Guidance on Web Accessibility and the ADA_DOJ. <https://beta.ada.gov/resources/web-guidance/>
- [35] 2022. Moz's list of the most popular 500 websites on the internet. <https://moz.com/top500>. Accessed: 2022-03-14.
- [36] 2022. Tenon.io. <https://tenon.io/>. Accessed: 2022-03-17.
- [37] 2022. The WebAIM Million: An annual accessibility analysis of the top 1,000,000 home pages. <https://webaim.org/projects/million/>. Updated: 2022-03-10.
- [38] 2022. Understanding Success Criterion 1.4.3: Contrast (Minimum): Relative Luminance. <https://www.w3.org/WAI/WCAG21/Understanding/contrast-minimum.html#dfn-relative-luminance>. Updated: 2022-04-13.
- [39] 2022. Understanding Success Criterion 2.4.11: Focus Appearance. <https://www.w3.org/WAI/WCAG22/Understanding/focus-appearance.html>. Updated: 2022-09-15.
- [40] 2022. Understanding Success Criterion 2.4.12: Focus Not Obscured (Minimum). <https://www.w3.org/WAI/WCAG22/Understanding/focus-not-obscured-minimum.html>. Updated: 2022-09-15.
- [41] 2022. Understanding Success Criterion 2.4.13: Focus Not Obscured (Enhanced). <https://www.w3.org/WAI/WCAG22/Understanding/focus-not-obscured-enhanced.html>. Updated: 2022-09-15.
- [42] 2022. W3C: Understanding Success Criterion 1.3.1: Info and Relationships. <https://www.w3.org/WAI/WCAG21/Understanding/info-and-relationships>. Updated: 2022-07-06.
- [43] 2022. W3C: Understanding Success Criterion 1.3.2: Meaningful Sequence. <https://www.w3.org/WAI/WCAG21/Understanding/meaningful-sequence>. Updated: 2022-07-06.
- [44] 2022. W3C: Understanding Success Criterion 2.1.1: Keyboard. <https://www.w3.org/WAI/WCAG21/Understanding/keyboard>. Updated: 2022-07-06.
- [45] 2022. W3C: Understanding Success Criterion 2.1.2: No Keyboard Trap. <https://www.w3.org/WAI/WCAG21/Understanding/no-keyboard-trap>. Updated: 2022-07-06.
- [46] 2022. W3C: Understanding Success Criterion 2.4.3: Focus Order. <https://www.w3.org/WAI/WCAG21/Understanding/focus-order>. Updated: 2022-07-06.
- [47] 2022. W3C: Understanding Success Criterion 2.4.7: Focus Visible. <https://www.w3.org/WAI/WCAG21/Understanding/focus-visible>. Updated: 2022-07-06.
- [48] 2022. W3C: Understanding Success Criterion 3.2.1: On Focus. <https://www.w3.org/WAI/WCAG21/Understanding/on-focus>. Updated: 2022-07-06.
- [49] 2022. W3C: Understanding Success Criterion 3.2.2: On Input. <https://www.w3.org/WAI/WCAG21/Understanding/on-input>. Updated: 2022-07-06.
- [50] 2022. Web Content Accessibility Guidelines (WCAG) 2.2: Success Criterion 2.4.11 Focus Appearance (Minimum). <https://www.w3.org/TR/WCAG22/#focus-appearance-minimum>. Updated: 2022-03-24.
- [51] 2023. BAGEL Project Web Site. <https://sites.google.com/usc.edu/bagel/home>. Updated: 2023-01-20.
- [52] Abdulmajeed Alameer, Sonal Mahajan, and William G. J. Halfond. 2016. Detecting and Localizing Internationalization Presentation Failures in Web Applications. In *2016 IEEE International Conference on Software Testing, Verification and Validation (ICST)*. 202–212. <https://doi.org/10.1109/ICST.2016.36>
- [53] Ali S. Alotaibi, Paul T. Chiou, and William G. J. Halfond. 2021. Automated Repair of Size-Based Inaccessibility Issues in Mobile Applications. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 730–742. <https://doi.org/10.1109/ASE51524.2021.9678625>
- [54] Abdulaziz Alshayban, Iftekhar Ahmed, and Sam Malek. 2020. Accessibility Issues in Android Apps: State of Affairs, Sentiments, and Ways Forward. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (Seoul, South Korea) (ICSE '20)*. Association for Computing Machinery, New York, NY, USA, 1323–1334. <https://doi.org/10.1145/3377811.3380392>
- [55] Mohammad Bajammal and Ali Mesbah. 2021. Semantic Web Accessibility Testing via Hierarchical Visual Analysis. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. 1610–1621. <https://doi.org/10.1109/ICSE43902.2021.00143>
- [56] Jeffrey P. Bigham. 2014. Making the Web Easier to See with Opportunistic Accessibility Improvement. In *Proceedings of the 27th Annual ACM Symposium on User Interface Software and Technology (Honolulu, Hawaii, USA) (UIST '14)*. Association for Computing Machinery, New York, NY, USA, 117–122. <https://doi.org/10.1145/2642918.2647357>
- [57] Syed Masum Billah, Vikas Ashok, Donald E. Porter, and IV. Ramakrishnan. 2018. SteeringWheel: A Locality-Preserving Magnification Interface for Low Vision Web Browsing. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems (Montreal QC, Canada) (CHI '18)*. Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3173574.3173594>
- [58] Peter Blanck. [n. d.]. ADA Title III and Web Equality: Litigation Begins. *eQuality* ([n. d.]), 81–104. <https://doi.org/10.1017/cbo9781107280151.009>
- [59] P. Blenkhorn, D.G. Evans, and A. Baude. 2002. Full-screen magnification for windows using DirectX overlays. *IEEE Transactions on Neural Systems and Rehabilitation Engineering* 10, 4 (2002), 225–231. <https://doi.org/10.1109/TNSRE.2002.806835>
- [60] Paul Blenkhorn and David Gareth Evans. 2006. A Screen Magnifier Using "High Level" Implementation Techniques. *IEEE Transactions on Neural Systems and Rehabilitation Engineering* 14, 4 (2006), 501–504. <https://doi.org/10.1109/TNSRE.2006.886728>
- [61] Deng Cai, S. Yu, Ji-Rong Wen, and W. Ma. 2003. VIPS: a Vision-based Page Segmentation Algorithm.
- [62] Stuart K. Card and David Nation. 2002. Degree-of-Interest Trees: A Component of an Attention-Reactive User Interface. In *Proceedings of the Working Conference on Advanced Visual Interfaces (Trento, Italy) (AVI '02)*. Association for Computing Machinery, New York, NY, USA, 231–245. <https://doi.org/10.1145/1556262.1556300>
- [63] Paul T. Chiou, Ali S. Alotaibi, and William G. J. Halfond. 2021. Detecting and Localizing Keyboard Accessibility Failures in Web Applications. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Athens, Greece) (ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 855–867. <https://doi.org/10.1145/3468264.3468581>
- [64] Mick Couper, Reg Baker, and Joanne Mechling. 2011. Placement and Design of Navigation Buttons in Web Surveys. *Survey Practice* 4 (02 2011), 1–11. <https://doi.org/10.29115/SP-2011-0001>
- [65] Nádia Fernandes, Daniel Costa, Sergio Neves, Carlos Duarte, and Luís Carriço. 2012. Evaluating the accessibility of rich internet applications. In *Proceedings of the International Cross-Disciplinary Conference on Web Accessibility (W4A '12)*. Association for Computing Machinery, Lyon, France, 1–4. <https://doi.org/10.1145/2207016.2207019>

- [66] Leah Findlater, Karyn Moffatt, Joanna McGrenere, and Jessica Dawson. 2009. Ephemeral adaptation: The use of gradual onset to improve menu selection performance. *Conference on Human Factors in Computing Systems - Proceedings*, 1655–1664. <https://doi.org/10.1145/1518701.1518956>
- [67] Julie Fraser and Carl Gutwin. 2000. A Framework of Assistive Pointers for Low Vision Users. In *Proceedings of the Fourth International ACM Conference on Assistive Technologies* (Arlington, Virginia, USA) (Assets '00). Association for Computing Machinery, New York, NY, USA, 9–16. <https://doi.org/10.1145/354324.354329>
- [68] George W. Furnas. 1997. Effective View Navigation. In *Proceedings of the ACM SIGCHI Conference on Human Factors in Computing Systems* (Atlanta, Georgia, USA) (CHI '97). Association for Computing Machinery, New York, NY, USA, 367–374. <https://doi.org/10.1145/258549.258800>
- [69] Krzysztof Z. Gajos, Jacob O. Wobbrock, and Daniel S. Weld. 2007. Automatically Generating User Interfaces Adapted to Users' Motor and Vision Capabilities. In *Proceedings of the 20th Annual ACM Symposium on User Interface Software and Technology* (Newport, Rhode Island, USA) (UIST '07). Association for Computing Machinery, New York, NY, USA, 231–240. <https://doi.org/10.1145/1294211.1294253>
- [70] Yves Guiard, Renaud Blanch, and Michel Beaudouin-Lafon. 2004. Object Pointing: A Complement to Bitmap Pointing in GUIs. In *Proceedings of Graphics Interface 2004* (London, Ontario, Canada) (GI '04). Canadian Human-Computer Communications Society, Waterloo, CAN, 9–16.
- [71] Carl Gutwin. 2002. Improving Focus Targeting in Interactive Fisheye Views. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Minneapolis, Minnesota, USA) (CHI '02). Association for Computing Machinery, New York, NY, USA, 267–274. <https://doi.org/10.1145/503376.503424>
- [72] Johannes Harms, Christoph Wimmer, Karin Kappel, and Thomas Grechenig. 2014. Design Space for Focus+context Navigation in Web Forms. In *Proceedings of the 2014 ACM SIGCHI Symposium on Engineering Interactive Computing Systems* (Rome, Italy) (EICS '14). Association for Computing Machinery, New York, NY, USA, 39–44. <https://doi.org/10.1145/2607023.2610272>
- [73] Kristina Höök and Martin Svensson. 1999. *Evaluating Adaptive Navigation Support*. Springer London, London, 237–249. https://doi.org/10.1007/978-1-4471-0837-5_13
- [74] Kasper Hornbæk and Erik Frøkjær. 2001. Reading of Electronic Documents: The Usability of Linear, Fisheye, and Overview+detail Interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Seattle, Washington, USA) (CHI '01). Association for Computing Machinery, New York, NY, USA, 293–300. <https://doi.org/10.1145/365024.365118>
- [75] Wan Abdul Rahim Wan Mohd Isa, Ahmad Iqbal Hakim Suhaimi, Nadhirah Arifrn, Nurul Fatimah Ishak, and Nadilah Mohd Ralim. 2016. Accessibility evaluation using Web Content Accessibility Guidelines (WCAG) 2.0. In *2016 4th International Conference on User Science and Engineering (i-USER)*, 1–4. <https://doi.org/10.1109/IUSER.2016.7857924>
- [76] Caroline Jarrett and Gerry Gaffney. 2010. *Forms that work: designing Web forms for usability*. Elsevier/Morgan Kaufmann.
- [77] Richard L. Kline and Ephraim P. Glinert. 1995. Improving GUI Accessibility for People with Low Vision. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Denver, Colorado, USA) (CHI '95). ACM Press/Addison-Wesley Publishing Co., USA, 114–121. <https://doi.org/10.1145/223904.223919>
- [78] John Lamping, Ramana Rao, and Peter Pirolli. 1995. A Focus+context Technique Based on Hyperbolic Geometry for Visualizing Large Hierarchies. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Denver, Colorado, USA) (CHI '95). ACM Press/Addison-Wesley Publishing Co., USA, 401–408. <https://doi.org/10.1145/223904.223956>
- [79] Bing Liu. 2011. *Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data* (2 ed.). Springer-Verlag, Berlin Heidelberg. <https://doi.org/10.1007/978-3-642-19460-3>
- [80] I. Scott MacKenzie. 1992. Fitts' Law as a Research and Design Tool in Human-Computer Interaction. *Hum.-Comput. Interact.* 7, 1 (March 1992), 91–139. https://doi.org/10.1207/s15327051hci0701_3
- [81] S. Mahajan, A. Alameer, P. McMinn, and W. G. J. Halfond. 2018. Automated Repair of Internationalization Presentation Failures in Web Pages Using Style Similarity Clustering and Search-Based Techniques. In *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*, 215–226.
- [82] S. Mahajan, K. B. Gadde, A. Pasala, and W. G. J. Halfond. 2016. Detecting and Localizing Visual Inconsistencies in Web Applications. In *2016 23rd Asia-Pacific Software Engineering Conference (APSEC)*, 361–364. <https://doi.org/10.1109/APSEC.2016.060>
- [83] Sonal Mahajan and William G.J. Halfond. 2014. Finding HTML Presentation Failures Using Image Comparison Techniques. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering* (Vasteras, Sweden) (ASE '14). Association for Computing Machinery, New York, NY, USA, 91–96. <https://doi.org/10.1145/2642937.2642966>
- [84] S. Mahajan and W. G. J. Halfond. 2015. Detection and Localization of HTML Presentation Failures Using Computer Vision-Based Techniques. In *2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST)*, 1–10. <https://doi.org/10.1109/ICST.2015.7102586>
- [85] S. Mahajan and W. G. J. Halfond. 2015. WebSee: A Tool for Debugging HTML Presentation Failures. In *2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST)*, 1–8. <https://doi.org/10.1109/ICST.2015.7102638>
- [86] S. Mahajan, B. Li, P. Behnamghader, and W. G. J. Halfond. 2016. Using Visual Symptoms for Debugging Presentation Failures in Web Applications. In *2016 IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 191–201. <https://doi.org/10.1109/ICST.2016.35>
- [87] Uwe Malinowski. 1993. Adjusting the Presentation of Forms to Users' Behavior. In *Proceedings of the 1st International Conference on Intelligent User Interfaces* (Orlando, Florida, USA) (IUI '93). Association for Computing Machinery, New York, NY, USA, 247–249. <https://doi.org/10.1145/169891.170016>
- [88] Pavel Panchekha, Michael D. Ernst, Zachary Tatlock, and Shoaib Kamil. 2019. Modular verification of web page layout. *Proceedings of the ACM on Programming Languages* 3, OOPSLA (Oct. 2019), 151:1–151:26. <https://doi.org/10.1145/3360577>
- [89] Pavel Panchekha, Adam T. Geller, Michael D. Ernst, Zachary Tatlock, and Shoaib Kamil. 2018. Verifying that web pages have accessible layout. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2018)*. Association for Computing Machinery, Philadelphia, PA, USA, 1–14. <https://doi.org/10.1145/3192366.3192407>
- [90] Iñigo Perona, Ainhoa Yera, Olatz Arbelaitz, Javier Muguerza, J. Eduardo Pérez, and Xabier Valencia. 2019. Towards automatic problem detection in web navigation based on client-side interaction data. In *Proceedings of the XX International Conference on Human Computer Interaction (Interacción '19)*. Association for Computing Machinery, Donostia, Gipuzkoa, Spain, 1–4. <https://doi.org/10.1145/3335595.3335642>
- [91] C. Pilgrim. 2012. Website Navigation Tools - A Decade of Design Trends 2002 to 2011. In *AUIC*.
- [92] Navid Salehnamadi, Abdulaziz Alshayban, Jun-Wei Lin, Iftekhar Ahmed, Stacy Branham, and Sam Malek. 2021. *Latte: Use-Case and Assistive-Service Driven Automated Accessibility Testing Framework for Android*. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3411764.3445455>
- [93] Andrés Sanoja and Stéphane Gançarski. 2014. Block-o-Matic: A web page segmentation framework. In *2014 International Conference on Multimedia Computing and Systems (ICMCS)*, 595–600. <https://doi.org/10.1109/ICMCS.2014.6911249>
- [94] Teija Vainio. 2010. A Review of the Navigation HCI Research During the 2000's. *International Journal of Interactive Mobile Technologies (iJIM)* (07 2010). <https://doi.org/10.3991/ijim.v4i3.1270>
- [95] William Massami Watanabe, Renata P. M. Fortes, and Ana Luiza Dias. 2012. Using acceptance tests to validate accessibility requirements in RIA. In *Proceedings of the International Cross-Disciplinary Conference on Web Accessibility (W4A '12)*. Association for Computing Machinery, Lyon, France, 1–10. <https://doi.org/10.1145/2207016.2207022>
- [96] Yangli Hector Yee and Anna Newman. 2004. A Perceptual Metric for Production Testing. In *ACM SIGGRAPH 2004 Sketches* (Los Angeles, California) (SIGGRAPH '04). Association for Computing Machinery, New York, NY, USA, 121. <https://doi.org/10.1145/1186223.1186374>
- [97] Gabriel Ytterberg. [n.d.]. The Median Cut Algorithm for Color Quantization. https://medium.com/@gytterberg_14295/the-median-cut-algorithm-for-color-quantization-cc1128a0c534. Updated: 2021-07-23.
- [98] Yanhong Zhai and Bing Liu. 2005. Web Data Extraction Based on Partial Tree Alignment. In *Proceedings of the 14th International Conference on World Wide Web* (Chiba, Japan) (WWW '05). Association for Computing Machinery, New York, NY, USA, 76–85. <https://doi.org/10.1145/1060745.1060761>
- [99] Zhengxuan Zhao, Pei-Luen Patrick Rau, Ting Zhang, and Gavriel Salvendy. 2009. Visual search-based design and evaluation of screen magnifiers for older and visually impaired users. *International Journal of Human-Computer Studies* 67, 8 (2009), 663–675. <https://doi.org/10.1016/j.ijhcs.2009.03.006>
- [100] Yu Zhong, Astrid Weber, Casey Burkhardt, Phil Weaver, and Jeffrey P. Bigham. 2015. Enhancing Android Accessibility for Users with Hand Tremor by Reducing Fine Pointing and Steady Tapping. In *Proceedings of the 12th International Web for All Conference* (Florence, Italy) (W4A '15). Association for Computing Machinery, New York, NY, USA, Article 29, 10 pages. <https://doi.org/10.1145/2745555.2747277>
- [101] Jun Zhu, Zaiqing Nie, Ji-Rong Wen, Bo Zhang, and Wei-Ying Ma. 2006. Simultaneous Record Detection and Attribute Labeling in Web Data Extraction. In *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Philadelphia, PA, USA) (KDD '06). Association for Computing Machinery, New York, NY, USA, 494–503. <https://doi.org/10.1145/1150402.1150457>
- [102] Ali S. Alotaibi, Paul T. Chiou, and William G.J. Halfond. 2022. Automated Detection of TalkBack Interactive Accessibility Failures in Android Applications. In *2022 IEEE 15th International Conference on Software Testing, Verification and Validation (ICST)*, 232–243. <https://doi.org/10.1109/ICST53961.2022.00033>